

Assignment 8

Task 1 MicroJava byte code (while)

Determine the MicroJava byte code implementing the following statement sequence. Assume that i is a local variable at address 2, and g is a global variable at address 3. For each MJVM instruction, additionally represent its code address (starting at 0), instruction length (in bytes), and the resulting content of the expression stack at run time (for the first loop iteration)

```
g = 5;
i = 2;
while (i < 10) {
    g = g + i * i;
    i++;
}
```

Theoretical Introduction: The Expression Stack (EStack)

The MicroJava Virtual Machine (μ JVM) is designed as a pure **stack machine**. Unlike register-based architectures, it does not utilize general-purpose registers for storing intermediate values or evaluating expressions. Instead, all operations are executed via a central runtime data structure known as the **Expression Stack (EStack)**.

Data is manipulated based on the Last-In, First-Out (LIFO) principle:

- **Implicit Operands:** Arithmetic and logical instructions (such as `add`, `sub`, `mul`) do not specify explicit register or memory operands. They implicitly pop the required number of arguments from the top of the EStack, perform the operation, and push the resulting value back onto the stack.
- **Load/Store Operations:** Values are explicitly transferred from local variable slots or global memory spaces onto the EStack using specific load instructions, and returned via store instructions.
- **Post-Statement Invariant:** At the completion of each full high-level statement, the expression stack must return to an empty state.

Instruction Length Reference Table

The following table defines the specific byte sizes and layout requirements for the standard μ JVM instructions utilized within this task:

Table 1: Instruction Sizes and Descriptions of the μ JVM

Instruction	Description	Length (Bytes)	Operands
<code>constn</code>	Load constant n ($0 \leq n \leq 5$)	1	None (implicit)
<code>const</code>	Load arbitrary 4-byte constant	5	4-byte value
<code>loadn</code>	Load local variable from index n ($0 \leq n \leq 3$)	1	None (implicit)
<code>storen</code>	Store top stack element to local variable n ($0 \leq n \leq 3$)	1	None (implicit)
<code>getstatic</code>	Load global variable from static address	3	2-byte address
<code>putstatic</code>	Store top stack element to static address	3	2-byte address
<code>mul</code>	Multiply two topmost stack elements	1	None
<code>add</code>	Add two topmost stack elements	1	None
<code>inc</code>	Increment local variable by a literal byte value	3	1-byte index, 1-byte value
<code>jmp</code>	Unconditional relative jump	3	2-byte signed offset
<code>jcc</code>	Conditional relative jump (e.g., <code>jge</code> , <code>jle</code>)	3	2-byte signed offset

Byte Code Generation and Runtime Trace

Below is the complete sequence of generated MicroJava byte code instructions corresponding to the statement sequence. The local variable i is mapped to address 2, and the global variable g is located at address 3. The **EStack** column represents the execution state during the first iteration of the loop.

```
g = 5;
i = 2;
while (i < 10) {
    g = g + i * i;
    i++;
}
```

Address	Instruction	Bytes	Estack
0	const5	1	5
1	putstatic 3	3	-
4	const2	1	2
5	store2	1	-
6	load2	1	2
7	const 10	5	2,10
12	jge +19 (= 31)	3	-
15	gstatic 3	3	5
18	load2	1	5, 2
19	load2	1	5, 2, 2
20	mul	1	5, 4
21	add	1	9
22	putstatic 3	3	-
25	inc 2,1	3	-
28	jmp -22 (= 6)	3	-
31			

Address

g ... 3

i ... 2

Jump Offset Calculations

1. **Conditional Exit Loop (jge):** The loop terminates at the address immediately following the backward jump, which is address 31. The conditional jump instruction starts at address 12. The relative offset is calculated as:

$$\text{Offset} = \text{Target Address} - \text{Instruction Start Address} = 31 - 12 = +19$$

2. **Unconditional Loop Back (jmp):** The loop must branch back to the loop condition evaluation, which starts at address 6. The jump instruction itself resides at address 28. The relative offset is calculated as:

$$\text{Offset} = \text{Target Address} - \text{Instruction Start Address} = 6 - 28 = -22$$

Using auxiliary methods in class *Code*

```
class Code {  
    private static final int eq = 0, ne = 1, lt = 2, le = 3, gt = 4, ge = 5;  
    private static int inverse[] = {ne, eq, ge, gt, le, lt};  
    ...  
    public static void tJump (int op, Label label) {  
        put(jcc + op); // jeq, jne, jlt, jle, jgt, jge  
        label.putAdr();  
    }  
  
    public static void fJump (int op, Label label) {  
        put(jcc + inverse[op]); // jne, jeq, jge, jgt, jle, jlt  
        label.putAdr();  
    }  
    ...  
}
```

Usage

```
"if" "(" Condition <↑x> ")"      (. Code.fJump(x.op, x.fLabel); .)
```

Figure 1: tJump and fJump, slide 67

Task 2 MicroJava Bytecode for if - else

Give the MicroJava byte code implementing the following statement sequence. Assume that *x* and *y* are local variables at addresses 0 and 1, respectively. For each MJVM instruction, additionally represent its code address (starting at 0), instruction length (in bytes), and the resulting content of the expression stack at run time, assuming an initial value of 9 for *x*. Mark all instructions that will NOT be executed for this value of *x*.

Theoretical Background

This task involves translating a multi-branch conditional statement into MicroJava bytecode. Conditions are evaluated using the expression stack and jump instructions such as `jle`, `jeq`, and `jmp`. Each conditional block is skipped or executed based on comparison outcomes and requires careful control of jump destinations to maintain correct program flow.

To translate a high-level conditional branch:

- The operands of the condition are loaded onto the expression stack (EStack).
- An **inverse conditional jump** operator is selected to bypass the immediate block if the condition evaluates to false, jumping directly to the subsequent `else-if` or `else` block.
- At the end of a conditional block, an unconditional jump (`jmp`) is required to skip all alternative branches and reach the end of the statement sequence.

The statement (probably the Collatz conjecture or hailstone sequence):

```
if (x <= 1) y = 1;  
else if (x % 2 == 0) y = x / 2;  
else y = x * 3 + 1;
```

Byte Code Generation

The table below illustrates the generated μ JVM byte code for the conditional statement sequence. Local variable x is stored at address 0, and local variable y is stored at address 1. The execution trace and EStack state are calculated based on an initial runtime value of $x = 9$.

Addresses:

$x \dots 0$

$y \dots 1$

Address	Instruction	Bytes	Estack	Status	Comment
0	load0	1	9	executed	
1	const1	1	9,1	executed	
2	jgt +8	3	-	executed	Condition: $9 \leq 1$ false -> jump taken
5	const1	1	-	not executed	$y = 1$ block skipped
6	store1	1	-	not executed	
7	jmp +23	3	-	not executed	
10	load0	1	9	executed	else if condition start
11	const2	1	9,2	executed	
12	rem	1	1	executed	computes $9 \bmod 2$
13	const0	1	1,0	executed	
14	jne +10	3	-	executed	condition $1 == 0$ is false -> jump taken
17	load0	1	-	not executed	$y = x / 2$ block skipped
18	const2	1	-	not executed	
19	div	1	-	not executed	
20	store1	1	-	not executed	
21	jmp +9	3	-	not executed	
24	load0	1	9	executed	else block start
25	const3	1	9,3	executed	
26	mul	1	27	executed	computes $9 * 3$
27	const1	1	27,1	executed	
28	add	1	28	executed	computes $27 + 1$
29	store1	1	-	executed	stores 28 into y
30					

Relative Jump Computations

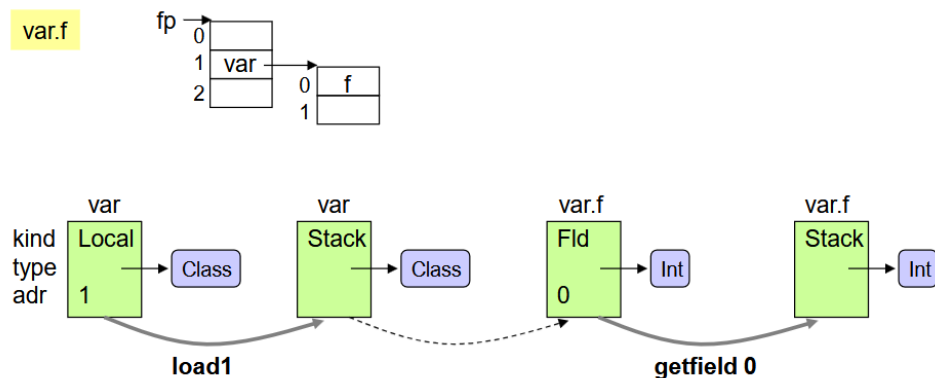
1. **First Branch** jgt +8: The jump targets the else-if block starting at address 10. The instruction itself is located at address 2.
2. **Second Branch** jne +10: The jump targets the final fallback else block at address 24. The instruction starts at address 14.

Task 3 Operand descriptors – object fields

Consider the expression `d.stat.avg`, where `d` is a local variable at address 0 pointing to a `Dataset` object on the heap. The `Dataset` class has been declared as follows::

```
class Stat { int min, max, avg; }
class Dataset {
    int[] val;
    Stat stat;
}
```

(a) Represent the operand descriptors and generated MJVM instructions of the given expression as on lecture slide 42.

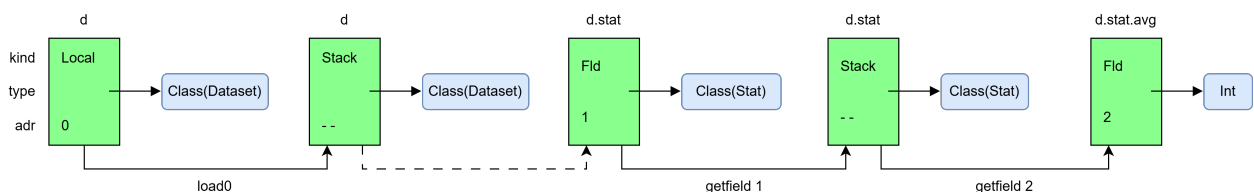


Introduction to Field Descriptors

During the code generation phase of a compiler, access chains to heap-allocated object fields are processed from left to right using an Attributed Translation Grammar (ATG). The compiler tracks variables and intermediate expressions via **Operand Descriptors**. An operand descriptor contains semantic metadata including the operand's kind (e.g., `Local`, `Fld`, `Stack`), its structural type, and its memory address or object field offset.

In accordance with the tasks of code generation outlined in the lecture (selecting appropriate instructions and managing the runtime stack), the compiler uses an Attributed Translation Grammar (ATG) to process designators from left to right. The internal compilation state is tracked using an **Operand Descriptor** (x), which bridges the compile-time metadata with the runtime Expression Stack (`EStack`) behavior as visually demonstrated on Slide 42.

Solution (a)



Step-by-Step Descriptor Transformation (`d.stat.avg`)

The sequential compilation of the expression chain `d.stat.avg` progresses through three distinct compiler and runtime states:

Step 1: Resolving the Base Identifier `d`

The compiler performs a symbol table lookup for the identifier `d`. It is identified as a local variable located at address 0.

- **Compiler Descriptor State (x):**

```
kind: Local
type: Class (Dataset)
adr: 0
```

- **Emitted Code:** *None* (Delayed code generation for local variables).
- **Runtime EStack State:** [] (Empty)

Step 2: Processing the First Field Access *.stat*

The ATG detects the field access operator. Because the current descriptor x .kind is *Local*, the compiler must force its base address onto the stack by invoking `Code.load( $x$ )`. Then, the field *stat* is resolved within the scope of class *Dataset* (Offset 1, as field *val* occupies offset 0).

- **Instruction Emitted by `Code.load( $x$ )`:**

```
load0
```

- **Temporary Descriptor State (Post-Load):** kind = *Stack*, type = *Class (Dataset)*
- **Runtime EStack State:** [`adr( $d$ )`]
- **Final Sub-Expression Descriptor State ($d.stat$):**

```
kind: Fld
type: Class (Stat)
adr: 1 (Field offset within Dataset)
```

Step 3: Processing the Second Field Access *.avg*

The next field access is processed. Since x .kind is currently *Fld*, loading the base address of the nested *Stat* sub-object from the heap is required. `Code.load( $x$ )` is called again. Finally, the field *avg* is resolved inside class *Stat* (Offset 2, since *min* = 0, *max* = 1).

- **Instruction Emitted by `Code.load( $x$ )`:**

```
getfield 1
```

- **Temporary Descriptor State (Post-Load):** kind = *Stack*, type = *Class (Stat)*
- **Runtime EStack State:** [`adr( $d.stat$ )`]
- **Final Designator Descriptor State ($d.stat.avg$):**

```
kind: Fld
type: Int
adr: 2 (Field offset within Stat)
```

*Note: When this designator is eventually evaluated as a factor within an expression, a final terminal `Code.load( $x$ )` call is triggered, emitting `getfield 2` and transitioning the descriptor to kind = *Stack* and type = *Int*.*

(b) Assume meaningful run-time values for addresses and content of the objects involved. Then represent instruction length (in bytes) and content of the expression stack for each MJVM instruction implementing the given expression.

To simulate the execution, the following hypothetical runtime memory mapping is assumed:

- Local variable slot 0 contains the heap pointer 1000 (base of object *d*).
- `Heap[1000 + 1]` holds the reference pointer to the *Stat* object, located at address 2500.
- `Heap[2500 + 2]` holds the primitive integer value 42 representing the field *avg*.

The complete step-by-step instruction execution trace and corresponding Expression Stack states are defined below:

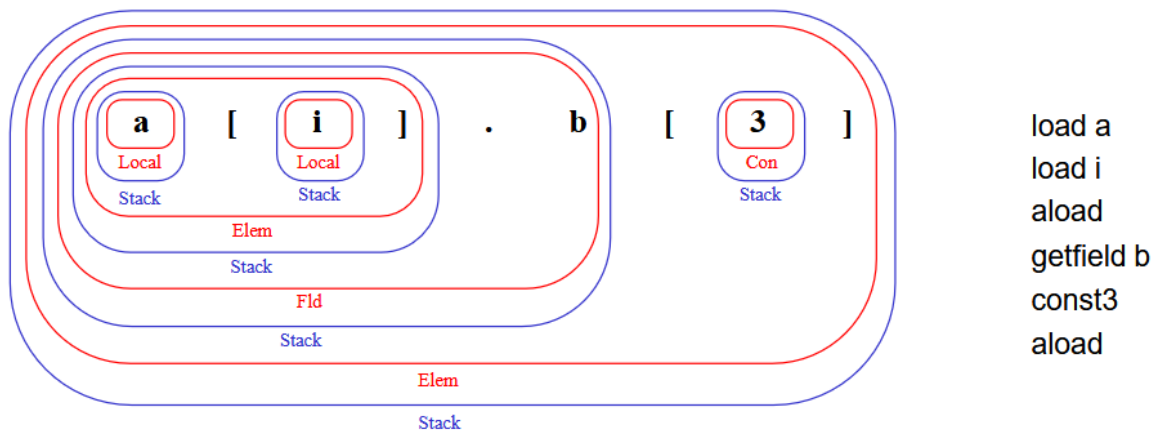
Table 2: Instruction Execution and Runtime Stack Trace

Address	Instruction	Length (Bytes)	EStack (Post-Execution)
0	load0	1	1000
1	getfield 1	3	2500
4	getfield 2	3	42

Task 4 Operand descriptors – array elements

Consider the expression `d.val[i+1]`, where `d` is the same local variable as in Task 3, and `i` is a local variable at address 1.

(a) Represent the operand descriptors and generated MJVM instructions of the given expression as on lecture slide 47.



Introduction to Nested Array Resolution

Following the operational model on Slide 47, expressions involving nested array elements combined with field selections are evaluated using an onion-skin layering architecture. Each structural level must transition into a runtime value on the stack before subsequent offsets or index operations can be appended. Array components are fundamentally tracked via the `Elem` descriptor kind.

The sequential compilation process governed by the ATG yields the following internal states and instruction listings:

Note: If evaluated as a terminal factor, a subsequent `Code.load(x)` invocation on this `Elem` descriptor emits the final `aload` array retrieval opcode.

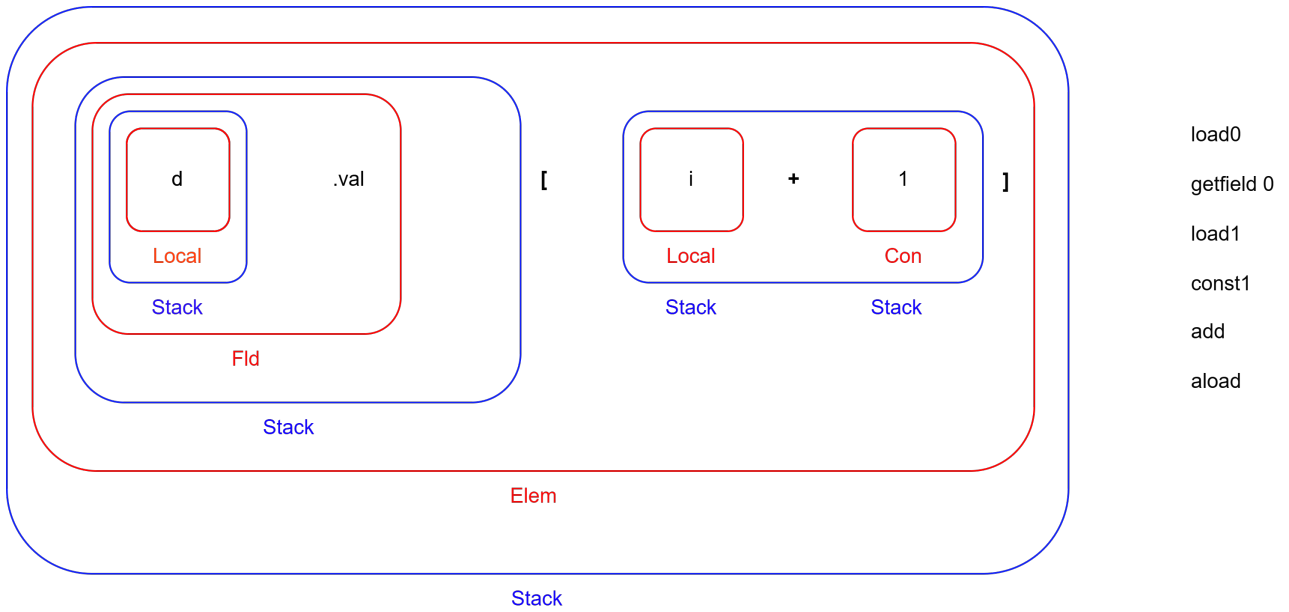
Solution (a)

(b) Assume meaningful run-time values for addresses and content of the objects involved. Then represent instruction length (in bytes) and content of the expression stack for each MJVM instruction implementing the given expression.

(a) Encapsulated Descriptor Nesting Analysis (Slide 47 Layout)

In strict alignment with the graphical notation scheme from Slide 47, the variable evaluations are encapsulated into geometric compiler boundaries (red) and runtime execution environments (blue) as specified below:

- **Core Layer (d):** Registered as `kind = Local`. Processing the structural dot selector invokes `Code.load(x)` → emits `load0` → transitions to `Stack`.



- **Field Layer (.val):** Resolves access fields within class layouts. Field `val` maps to word offset 0. Encapsulates into `kind = Fld`. To prepare for indexed lookups, the base pointer must be pushed to the stack → **emits** `getfield 0` → **transitions to** `Stack`.
- **Index Parameter Block ([i+1]):** Evaluates expression variables independently:
 1. Load index variable `i`: `kind = Local` → **emits** `load1` → **transitions to** `Stack`.
 2. Load index increment `1`: `kind = Con` → **emits** `const1` → **transitions to** `Stack`.
 3. Execute index addition: → **emits** `add`.
- **Element Container Layer (Elem):** The structural elements unify into an array component state: `kind = Elem`.
- **Terminal Expression Factor Phase:** Compiling the complete descriptor as an expression factor triggers the terminal array index load: → **emits** `aload` → **transitions to** `Stack`.

(b) Runtime Simulation and Instruction Trace

The trace models array pointer allocation address 3000 (extracted from `Heap[1000+0]`) along with a runtime value of 2 inside slot index 1 (`i`). The index equates to $2 + 1 = 3$, shifting structural execution to heap slot address $3000 + 1 + 3 = 3004$, containing element data 88.

Table 3: Instruction Execution Step Trace for `d.val[i+1]`

Address	Instruction	Instruction Length (Bytes)	EStack State
0	load0	1	1000
1	getfield 0	3	3000
4	load1	1	3000, 2
5	const1	1	3000, 2, 1
6	add	1	3000, 3
7	aload	1	88